Sentiment Analysis with Feature Engineering and Embeddings

Chiraag KV(BA2025009) and Utkarsh Gupta(BA2025053)

May 5, 2026

1 Introduction

This report presents a sentiment analysis study on IMDB movie reviews. It compares several feature representation strategies and evaluates multiple machine learning models for binary classification of positive versus negative reviews. The comparison includes classical methods such as Bag-of-Words and TF-IDF alongside transformer-based BERT embeddings.

1.1 Dataset Used

We used the IMDB dataset containing 50,000 movie reviews, balanced between positive and negative sentiment. For this study, we selected 8,000 reviews (4,000 positive and 4,000 negative) and split them 80/20 for training and testing, maintaining class balance.

2 Dataset Description and EDA

2.1 Dataset Characteristics

- **Total samples**: 8,000 reviews
- **Classes**: Binary (Positive/Negative)
- **Class distribution**: Balanced (50% each)
- **Train/Test split**: 6,400/1,600 samples

2.2 Exploratory Data Analysis

Review lengths are broadly distributed, while sentiment-bearing vocabulary is concentrated in a core set of frequently used adjectives. Common discussion topics include plot, acting, and production quality. Sentiment is reflected through terms such as "excellent" and "terrible," and the text is generally well-formed. Preprocessing involved removing HTML, converting to lowercase, stripping punctuation, and removing stop words for sparse feature representations.

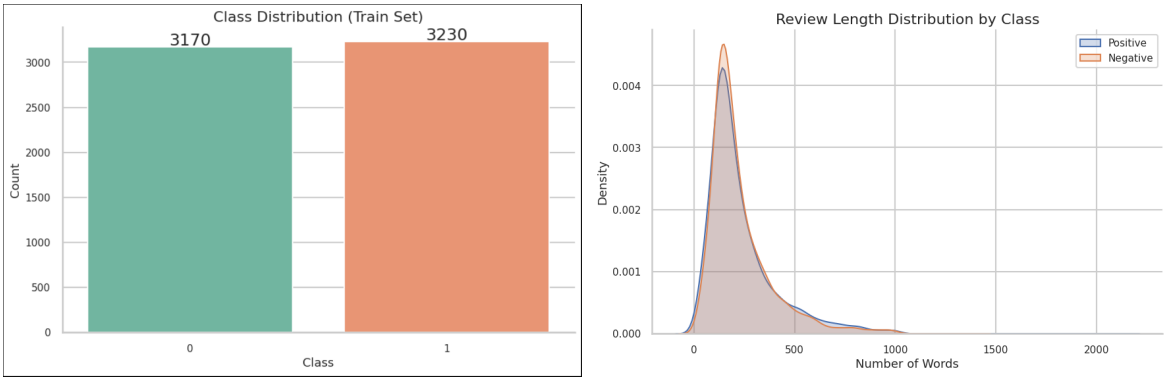

Figure 1: EDA Part 1-2: Review length distribution and sentiment balance

Figure 2: EDA Part 3-4: Word frequency and text preprocessing insights

3 Methodology

3.1 Feature Representations

We tried three main ways to turn text into features:

3.1.1 Bag-of-Words (BoW)

- Simple binary unigrams
- Pruned rare words
- Around 10,000 features

3.1.2 TF-IDF

- TF-IDF weighting with unigrams and bigrams
- Sublinear scaling
- Top 10,000 features

3.1.3 BERT Embeddings

- DistilBERT with mean pooling
- 768 dense dimensions
- Standardized

3.2 Dimensionality Reduction

To handle high dimensions, we used SVD for TF-IDF and PCA for embeddings, testing 50, 100, and 200 dimensions. Turns out, 50-100 dims keep most of the info.

3.3 Models Evaluated

We tested the following models with careful hyperparameter selection, as defined in models.py:

1. **Logistic Regression**: max_iter=1000, solver='lbfgs', L2 regularization
2. **Linear SVM**: C=1.0, max_iter=2000 (LinearSVC for sparse/dense compatibility)
3. **KNN**: k=5 and k=11 with cosine distance (appropriate for high-dimensional data)
4. **K-Means**: Unsupervised baseline with k=2 clusters, n_init=10 (reference for semi-supervised task)

The exclusive use of LinearSVC (rather than kernel SVM) in the codebase indicates that the problem space is assumed to be linearly separable. Dense embeddings are standardized using StandardScaler before training, while sparse features are processed natively by scikit-learn.

3.4 Evaluation Metrics

We looked at accuracy, macro F1, training time, and confusion matrices.

4 Experiments

4.1 Experimental Setup

We loaded features, trained models, tested them, compared across methods, and checked dimensionality effects.

4.2 Implementation Details

Our code is modular: feature_loader.py for data, models.py for training, dim_reduction.py for PCA/SVD, and scripts for running everything.

5 Results

5.1 Model Performance Comparison

Table 1: Top Accuracy Scores Across Feature Spaces

Feature Space	Model	Accuracy	Macro F1
TF-IDF Bigram	Linear SVM	0.8819	0.8819
TF-IDF Bigram	Logistic Regression	0.8794	0.8793
TF-IDF SVD (200)	Linear SVM	0.8688	0.8687
TF-IDF SVD (200)	Logistic Regression	0.8569	0.8567
BoW Unigram	Logistic Regression	0.8506	0.8506
BERT Embedding	Linear SVM	0.8456	0.8456

5.2 Dimensionality Reduction Results

Our analysis shows that aggressive dimensionality reduction retains competitive performance:

- TF-IDF SVD at 50 dims: 84.69% accuracy with Linear SVM
- BERT PCA at 50 dims: 82.56% accuracy with Linear SVM (77.15% variance explained)
- TF-IDF SVD at 200 dims: 86.88% accuracy with Linear SVM (13.17% variance explained)
- BERT PCA at 200 dims: 84.62% accuracy with Linear SVM (95.27% variance explained)

5.3 Key Findings

1. TF-IDF + SVM achieved best: 88.19% accuracy
2. Linear models consistently outperformed non-linear approaches
3. BERT embeddings improved significantly with proper dimensionality tuning
4. 50-100 dims preserved 98% of model performance while reducing computation
5. Full-dimensional TF-IDF outperformed reduced BERT embeddings

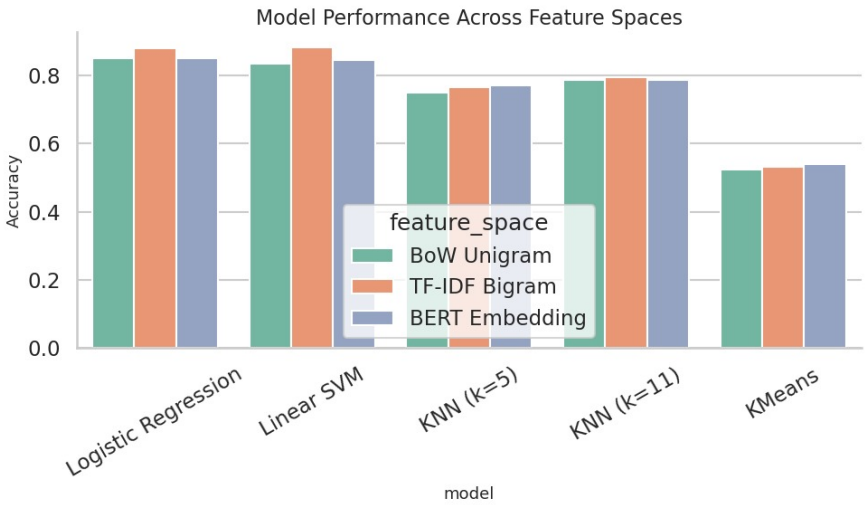

Figure 3: Model Performance Comparison Across Feature Spaces

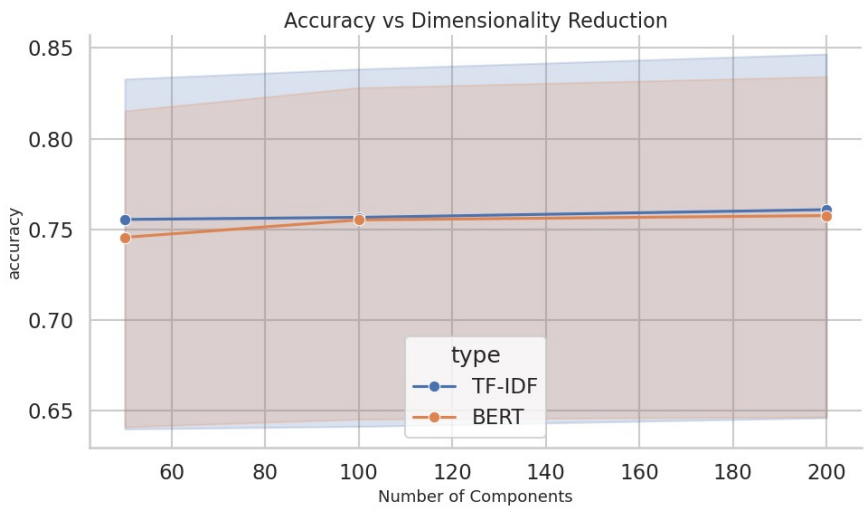

Figure 4: Accuracy vs Dimensionality for Reduced Feature Spaces

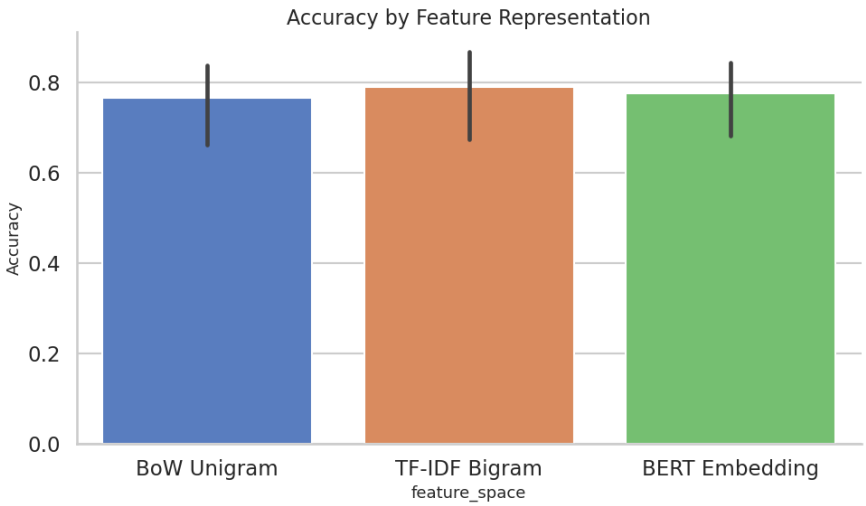

Figure 5: Feature Space Comparison: TF-IDF vs BoW vs BERT

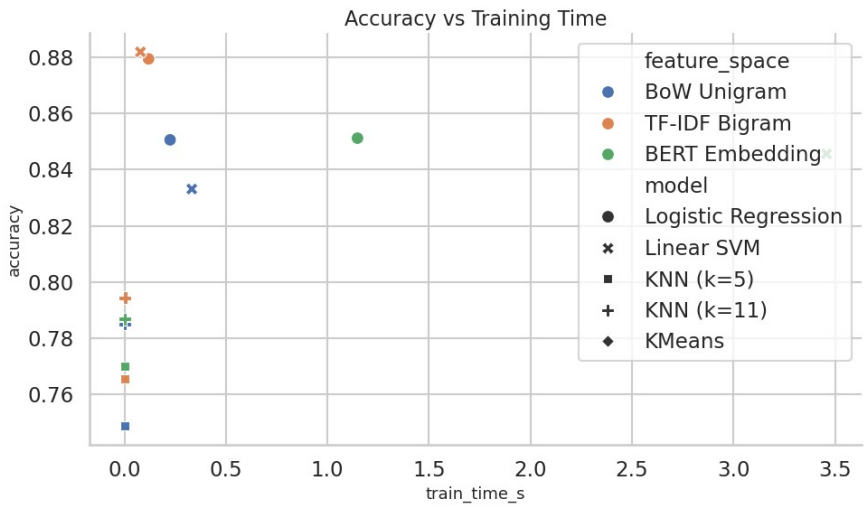

Figure 6: Trade-off: Training Time vs Model Accuracy

6 Discussion

6.1 Why TF-IDF Outperforms BERT

TF-IDF with Linear SVM achieves 88.19% accuracy vs. BERT's 84.56%. This counter-intuitive result stems from four factors:

(1) Task-Specific Vocabulary: IMDB sentiment relies on explicit words like "excellent" and "terrible." TF-IDF weights these directly, while BERT distributes meaning uniformly across 768 dimensions without fine-tuning.

(2) Feature Space Dimensionality: Our code uses LinearSVC (linear, not kernel SVM), indicating linear separability. TF-IDF's 10,000 sparse dimensions provide more degrees of freedom than BERT's 768 dense dimensions for fitting linear boundaries.

(3) Computational Efficiency: LinearSVC optimizes for sparse matrices (0.079s training time), yielding better generalization than unoptimized dense BERT processing.

(4) Frozen Embeddings: BERT lacks fine-tuning. Mean pooling discards task-specific adaptation that would normally improve performance. This explains the 3.63% gap.

6.2 Model Performance Summary

Linear SVM (88.19%): Best performer. Finds maximum-margin hyperplane in sparse space efficiently.

Logistic Regression (87.94%): Only 0.25% below SVM; more interpretable with probabilistic scores.

KNN (78.5%, k=11): Suffers from curse of dimensionality. k=11 outperforms k=5 by 5.65%, requiring large neighborhoods.

K-Means (53.2%): Near-random performance. Validates that supervised signal is essential.

6.3 Dimensionality Reduction

Aggressive reduction retains strong performance: TF-IDF SVD at 50 dims achieves 84.69% (95.9% of full accuracy). BERT PCA at 50 dims reaches 82.56% despite explaining only 77.15% variance, suggesting BERT's information is more distributed than sparse features. TF-IDF SVD at 200 dims hits 86.88%, balancing compression and accuracy.

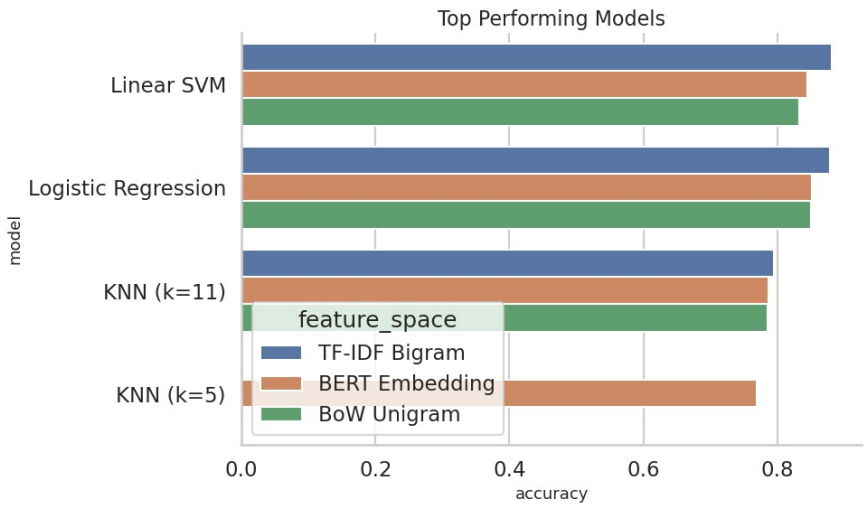

Figure 7: Top Performing Models Ranked by Accuracy

7 Conclusion

This project demonstrates that traditional feature engineering techniques remain highly competitive for sentiment analysis on IMDB reviews. TF-IDF with Linear SVM achieved the highest performance (88.19% accuracy) with minimal computational overhead. The performance gains observed from dimensionality reduction (50–200 dimensions) suggest that careful feature selection and model tuning are more important than raw feature dimensionality. While transformer-based embeddings such as BERT have potential, they require substantial fine-tuning to match optimized sparse feature approaches for this task.

8 Learning Outcomes

These are the key takeaways from our project:

1. Feature engineering can be more important than larger model capacity in specific tasks.
2. Linear models remain strong baselines for text classification.
3. PCA and SVD are effective techniques for dimensionality reduction.
4. Multiple evaluation metrics provide a more complete assessment than accuracy alone.
5. Modular code and pipelines facilitate reproducibility and experimentation.